

How Much Talk Does a Rescue Team Need? Decomposing the Value of Communication in Decentralized Multi-Agent Disaster Response

Aahil Sayed | Prachi Pawar | Purva Chaudhari | Sakshi Maher
124B1B062 | 124B1B283 | 124B1B291 | 124B1B298

Department of Computer Engineering, Pimpri Chinchwad College of Engineering (PCCOE), Pune, India
Year: TY Computer Engineering

Abstract—When several autonomous rescue units serve one disaster area, the cheapest way to lose time is for two of them to drive to the same victim. Coordination protocols prevent this, but they differ sharply in communication cost and centralization, and post-disaster networks are lossy. We ask which part of coordination actually buys the gain, and how the answer changes with team size and message loss. We extend RESQ-MAS, an open multi-agent rescue simulator, into a testbed of five allocation protocols of increasing communication cost: independent selection, claim broadcasting, one-round and iterative propose–resolve negotiation, and a centralized Hungarian matcher. Across 28,300 paired simulations on random worlds, one-round negotiation cuts mission completion time by 29.0% for two agents (1,000 worlds, Holm-corrected Wilcoxon $p < 10^{-4}$) and by up to 66% for eight. Three findings stand out. First, uncoordinated teams saturate: eight independent agents are only 1.7–2.5 $\times$ faster than one, against 5.0–6.7 $\times$ for negotiating teams, so the empirical price of anarchy grows from 1.3 to 3.1 with team size. Second, the value of negotiation over claim broadcasting is set by contention: 4% of the total gain at 15 victims per agent, 61% at 3. Third, iterative negotiation is statistically indistinguishable from the centralized matcher in all 15 team/workload configurations while sending 3–11 $\times$ less data, and under broadcast loss every protocol degrades smoothly, retaining roughly a $(1 - p)$ fraction of its gain. The testbed and all raw runs are released.

Index Terms—Multi-agent systems, multi-robot task allocation, disaster response, search and rescue, decentralized coordination, auction algorithms, Hungarian algorithm, price of anarchy, lossy communication, agent-based simulation.

I. INTRODUCTION

After an earthquake or flood, emergency operations centers dispatch ambulances, ground robots, and drones to many casualties at once, often over degraded radio links and with little time for central planning [1], [2]. A recurring and expensive failure mode in such operations is *duplicated effort*: two units independently decide that the same critical victim is the most urgent, both travel there, and one arrives to find nothing to do while other victims keep waiting. Multi-agent coordination research offers many remedies, from market-based auctions [3] to consensus-based and distributed optimization methods [4], [5] and centralized optimal assignment [6]. What a practitioner designing a dispatch system actually needs to know, however, is more specific: *which part* of coordination produces the

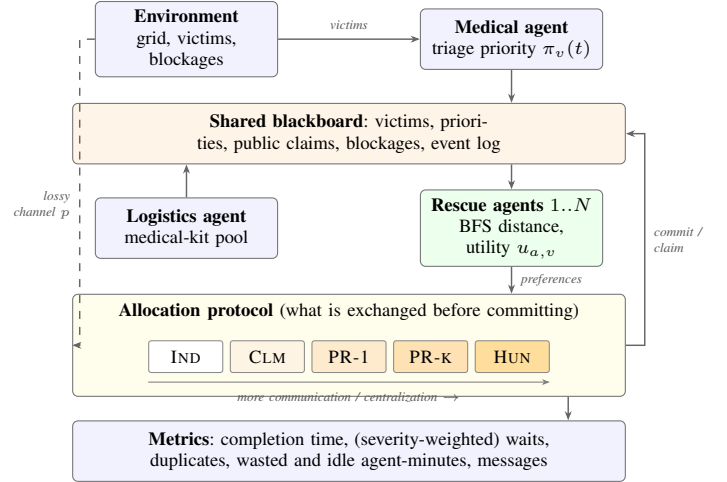


Fig. 1. RESQ-MAS testbed. The world is generated deterministically from a seed, so every protocol runs on an identical world (paired design). The medical agent publishes priorities to a shared blackboard; rescue agents score reachable victims and hand their preferences to one of five allocation protocols that differ only in what is communicated before an agent commits. Routing, replanning, pickup, and delivery are shared code, so any metric difference is attributable to the protocol alone.

benefit, *how much communication* it costs, and *how it behaves* when messages are lost.

This paper answers these questions with a controlled, paired simulation study. Our starting point is RESQ-MAS, an open-source disaster-rescue simulator in which a medical triage agent, a logistics agent, and a team of rescue agents interact through a shared blackboard. Its original release compared two policies on 20 hand-picked runs. We extend it into a protocol testbed (Fig. 1) with five allocation protocols ordered by communication cost, support for arbitrary team sizes, an explicit lossy-broadcast model, and instrumentation for wasted effort and message volume, and we evaluate it on 28,300 simulations over randomly generated worlds.

A. Contributions

- A **communication ladder** of five task-allocation protocols implemented on one shared agent codebase, so that the *only* difference between conditions is what agents

exchange before committing: nothing, public claims, one negotiation round, iterative negotiation, or full utility reports to a centralized optimal matcher.

- A **decomposition of the coordination gain** into a claim-broadcasting component and a same-tick negotiation component, showing that the share due to negotiation is governed by the contention ratio between agents and victims (4% to 61%).
- Evidence that **uncoordinated teams saturate**: adding agents without coordination yields at most $2.5\times$ speedup at eight agents, and the empirical price of anarchy grows roughly linearly with team size.
- Evidence that **decentralized iterative negotiation matches a centralized matcher** in all 15 tested configurations (no significant difference after Holm correction) with $3\text{--}11\times$ lower communication volume, and that all message-based protocols degrade gracefully under broadcast loss.
- A reproducible, open testbed with fixed seeds, paired statistics, and all raw runs released.

II. RELATED WORK

A. Multi-Robot Task Allocation

Recent surveys [6]–[8] organize multi-robot task allocation (MRTA) by whether a robot executes one or several tasks, whether a task needs one or several robots, and whether assignment is instantaneous or planned over time; Nunes *et al.* [9] extend this taxonomy to temporal and ordering constraints. Our setting, in which each agent serves one victim at a time and victims are known up front, is the single-task, single-robot, instantaneous-assignment class, repeated as agents become free. Each decision epoch is a linear assignment problem, solvable optimally by the Hungarian method, which Chopra *et al.* [5] showed can also be executed in a fully distributed way over a peer-to-peer network. Quinton *et al.* [3] review market-based MRTA, from Contract-Net-style single-item auctions to consensus-based bundle algorithms, and report that greedy auctions routinely come close to optimal allocations at a fraction of the communication cost. Rizk *et al.* [10] survey cooperative heterogeneous teams, including behaviour-based schemes that coordinate implicitly by broadcasting current activity, which corresponds to the claim-broadcasting rung of our ladder.

B. Coordination for Disaster Rescue

Queralta *et al.* [1] and Drew [2] survey multi-robot search-and-rescue systems and both single out coordination under unreliable communication as an open operational problem. Ghassemi and Chowdhury [11] formulate disaster-response MRTA with task deadlines and robot range and payload constraints and solve it with a decentralized bigraph-matching approach, showing that the allocation layer materially changes how many victims are served in time.

TABLE I
POSITIONING RELATIVE TO REVIEWED WORK

Study	Focus	Gap addressed here
Quinton [3], Chakraa [6] Chopra [5]	MRTA surveys (2023) Distributed Hungarian	Qualitative comparison; no controlled ladder of protocols Optimality with full exchange; cost of less talk not measured
Ghassemi [11] Otte [12]	Disaster MRTA with deadlines Auctions under loss	One method; no independent or claim-only baselines No decomposition of the gain or team-size scaling
Lott [14]	Allocators under degraded links (2026)	Consensus/auction family only; no rescue priorities or claim-only rung
This paper	Five-rung ladder, 28.3k paired runs	Decomposition, scaling law, optimality gap, loss robustness

C. Coordination under Limited Communication

Buckman *et al.* [4] extend the consensus-based bundle algorithm with partial replanning, trading convergence time for coordination quality when new tasks appear online. Otte *et al.* [12] compared auction variants under message loss and found that performance degrades gracefully as loss increases. Gielis *et al.* [13] review communication in multi-robot systems and argue that its cost and reliability should be treated as first-class design variables rather than assumed ideal. Most recently, Lott and Honary [14] benchmarked six decentralized allocators (CBAA, ACBBA, and others) on paired instances under Bernoulli, Gilbert–Elliott, and fading channels, comparing travel cost, reliability, and computation. On the game-theoretic side, Paccagnan *et al.* [15] characterize the price of anarchy, the ratio between the cost of uncoordinated equilibria and the social optimum, for distributed resource allocation, which gives a natural lens for measuring what coordination is worth.

D. Research Gap

Existing studies typically propose one protocol, or benchmark several allocators of the same family, so the *marginal* value of each additional kind of communication (none, public claims, negotiation, full centralization) is rarely isolated on one codebase. The scaling of that value with team size, the role of contention, and the cost of centralization are usually reported, if at all, for a single configuration. Table I positions this paper.

III. SYSTEM MODEL

A. Environment

The world is a $W \times W$ 4-connected grid ($W = 15$ or 20) in which about 10% of cells are buildings, one cell is a hospital, and two corner cells are depots from which rescue agents start. M victims are placed on random road cells with severities drawn from {critical, high, moderate, low} with probabilities {0.2, 0.3, 0.3, 0.2}. A fraction b of free road cells is scheduled to become blocked at a random time in the first 120 simulated minutes and to reopen 10–75 minutes later; blockages are published to all agents immediately and trigger

BFS replanning. One tick represents one simulated minute. A victim counts as rescued when delivered to the hospital; each rescue consumes one medical kit, allocated by the logistics agent, which refuses a second kit for a victim that already holds one. An agent whose target has been rescued by someone else, or whose kit request stays denied for 40 ticks, abandons that target.

B. Priorities and Utility

The medical agent republishes each waiting victim's priority as

$$\pi_v(t) = s_v + \lambda w_v(t), \quad (1)$$

where $s_v \in \{4, 3, 2, 1\}$ is the severity score, $w_v(t)$ the waiting time so far, and $\lambda = 0.1$, so severity dominates but long-waiting victims eventually rise. An idle agent a scores every reachable candidate victim v by

$$u_{a,v}(t) = \pi_v(t) - \beta d_a(v), \quad (2)$$

with $d_a(v)$ the BFS shortest-path length on the currently known grid and $\beta = 0.1$. All protocols use the same utility; they differ only in how preferences are turned into commitments.

C. The Communication Ladder

IND (independent). Each idle agent commits to its own $\arg \max_v u_{a,v}$ over all unrescued victims. Nothing is communicated, so agents may converge on the same victim or chase a victim already being carried.

CLM (claim broadcasting). After committing, an agent broadcasts a claim; other agents exclude claimed victims from their candidate sets. Choices made in the same tick are not negotiated, so simultaneous duplicates remain possible. This mirrors implicit coordination by activity broadcasting in behaviour-based teams [10].

PR-1 (one-round propose–resolve). Idle agents first broadcast a proposal $(v, u_{a,v})$. Proposals for the same victim are resolved deterministically: highest utility wins, ties go to shorter distance, then agent identifier. Winners commit and broadcast claims; losers stay idle for the rest of the tick and propose again at the next tick. The resolution rule is a pure function of public messages, so every agent can compute it locally; no privileged coordinator is needed. This is the original RESQ-MAS coordination mechanism.

PR-K (iterative propose–resolve). As PR-1, but losers immediately re-propose their best victim not yet won in the current tick, repeating until every idle agent holds a target or no candidate remains (Algorithm 1). This is a sequential single-item auction [3] executed within one tick.

HUN (centralized reference). Each idle agent reports its full utility row; a coordinator solves $\max \sum_{a,v} u_{a,v} x_{a,v}$ over one-to-one assignments with the Hungarian method as implemented in SciPy [16] and returns the matching. HUN is optimal *per decision epoch*, not over the whole mission, because it re-solves only for currently idle agents and never pre-empt's commitments.

Algorithm 1 PR-K: one tick of iterative propose–resolve

```

1:  $\mathcal{A} \leftarrow$  idle agents;  $\mathcal{T} \leftarrow$  victims won this tick
2: each  $a \in \mathcal{A}$  proposes  $v_a = \arg \max_{v \notin \text{claims} \cup \mathcal{T}} u_{a,v}$ 
3: while  $\mathcal{A} \neq \emptyset$  and some proposal exists do
4:   for each victim  $v$  with proposals do
5:     winner  $\leftarrow \arg \max(u_{a,v}, -d_a(v), -\text{id}_a)$ 
6:     winner commits, broadcasts claim;  $\mathcal{T} \leftarrow \mathcal{T} \cup \{v\}$ 
7:   end for
8:    $\mathcal{A} \leftarrow$  losers; each re-proposes over victims  $\notin \mathcal{T}$ 
9: end while

```

D. Lossy Broadcast Model

Every broadcast (proposal or claim) is independently lost with probability p for all receivers, a broadcast-erasure channel. A sender whose proposal is lost hears no objection and commits unilaterally; a lost claim leaves the victim visible as a candidate to others until its delivery to the hospital, which we assume is always observed. Loss draws use a random stream separate from world generation, so lossy and loss-free runs share the same world. At $p = 1$ every message-based protocol reduces exactly to IND.

E. Metrics

Per run we record the completion time T (minute the last victim is delivered), the mean waiting time of rescued victims, a severity-weighted waiting time $\bar{w}_s = \sum_v s_v w_v / \sum_v s_v$, the mean wait of critical victims, duplicate commitments (distinct victims to which two agents were simultaneously committed), wasted agent-minutes (time spent committed to targets later abandoned), idle agent-minutes while victims remained, and communication as broadcast count and payload (scalar values transmitted). From these we derive the speedup $S(N) = T(1)/T(N)$, the empirical price of anarchy $\text{PoA} = T_{\text{IND}}/T_{\text{HUN}}$, the optimality gap of protocol X , $(T_X - T_{\text{HUN}})/T_{\text{HUN}}$, and the *negotiation share* of the coordination gain,

$$\eta = \frac{T_{\text{CLM}} - T_{\text{PR-1}}}{T_{\text{IND}} - T_{\text{PR-1}}}, \quad (3)$$

the fraction of the total IND→PR-1 improvement that claim broadcasting alone does not deliver.

IV. EXPERIMENTAL DESIGN

All worlds are generated from seeds starting at 1000, disjoint from the curated demonstration seeds of the original release. Each world is run once under every protocol, so all comparisons are paired. Random building placement occasionally walls a victim or the hospital off from both depots; no protocol can rescue such a victim, so these worlds (19 of 1,019 at $W = 15$; 7–8 of 157–158 at $W = 20$) are excluded deterministically before any run and replaced by the next valid seed. Every retained run rescued every victim.

E1 – Generalization: $N = 2$, $M = 6$, $W = 15$, 1,000 worlds, $b \in \{0, 0.1\}$, all five protocols (10,000 runs). This is the original RESQ-MAS setting on random rather than curated worlds. **E2 – Scaling:** $N \in \{1, 2, 3, 4, 6, 8\}$, $M \in \{10, 20, 30\}$, $W = 20$, 150 worlds per cell

TABLE II
E1: TWO AGENTS, SIX VICTIMS, 1,000 RANDOM WORLDS (MEANS)

b	Prot.	T	Wait	$\bar{w}_s$	Crit.	Dup.	Waste	Msgs
0%	IND	99.3	54.4	45.8	25.2	4.07	69.9	0.0
0%	CLM	88.1	50.5	43.3	25.2	2.08	43.2	8.1
0%	PR-1	70.5	40.3	35.2	22.7	0.00	0.0	13.6
0%	PR-k	70.1	40.0	34.9	22.6	0.00	0.0	14.2
0%	HUN	69.9	39.9	34.9	22.7	0.00	0.0	18.1
10%	IND	100.6	55.0	46.2	25.4	4.09	70.7	0.0
10%	CLM	88.5	50.8	43.6	25.4	2.00	42.2	8.0
10%	PR-1	71.3	40.6	35.5	22.8	0.00	0.0	13.6
10%	PR-k	70.8	40.3	35.2	22.8	0.00	0.0	14.1
10%	HUN	70.6	40.2	35.1	22.9	0.00	0.0	18.1

All times in simulated minutes. $\bar{w}_s$: severity-weighted wait; Crit.: mean wait of critical victims; Dup.: duplicate commitments per run; Waste: agent-minutes on abandoned targets; Msgs: coordination broadcasts per run.

(11,700 runs). **E3 – Message loss:** $N = 4$, $M = 20$, $p \in \{0, 0.05, 0.1, 0.2, 0.3, 0.5, 0.7, 1\}$ for CLM, PR-1, PR-K (3,600 runs). **E4 – Blockages:** $N = 4$, $M = 20$, $b \in \{0, 0.1, 0.2, 0.3\}$, all protocols (3,000 runs).

Paired differences are tested with the two-sided Wilcoxon signed-rank test (SciPy [16]), with Holm’s step-down correction applied within each experiment family (36 tests in E1, 45 in E2). Effect sizes are matched-pairs rank-biserial correlations r_{rb} , and 95% confidence intervals for mean paired differences use 2,000 bootstrap resamples. The complete study (28,300 runs) takes about 3.5 minutes on four CPU cores.

V. RESULTS

A. E1: The Original Two-Agent Setting on Random Worlds

Table II and Fig. 2 summarize E1. On 1,000 random worlds, PR-1 reduces completion time from 99.3 to 70.5 minutes (29.0%), mean waiting time by 25.9%, and severity-weighted waiting time by 23.0%, and it eliminates duplicate commitments (4.07 per run to 0) and wasted effort (69.9 agent-minutes to 0). The original release reported a 20.9% completion-time gain on 20 curated runs; the curated benchmark therefore *understated* the effect rather than inflating it. Critical victims, the ones triage protects first under every protocol, still wait 10% less under negotiation (25.2 to 22.7 min), because agents are no longer tied up on duplicates when a critical victim becomes the best candidate.

Table III decomposes the gain. Claim broadcasting alone recovers 11.2 minutes; one-round negotiation adds a further 17.6. In this high-contention setting (three victims per agent), negotiation contributes 61% of the total. Beyond PR-1, returns vanish: PR-K adds 0.4 minutes and HUN a further 0.2. These last differences are statistically detectable with 1,000 pairs, but 91% of worlds are exact ties between PR-K and HUN and the effect size is small ($r_{rb} = 0.31$). Adding 10% blockages changes no conclusion.

B. E2: Uncoordinated Teams Saturate

Fig. 3 and Table IV show the central result. A single agent needs 293, 580, and 870 minutes for 10, 20, and 30 victims.

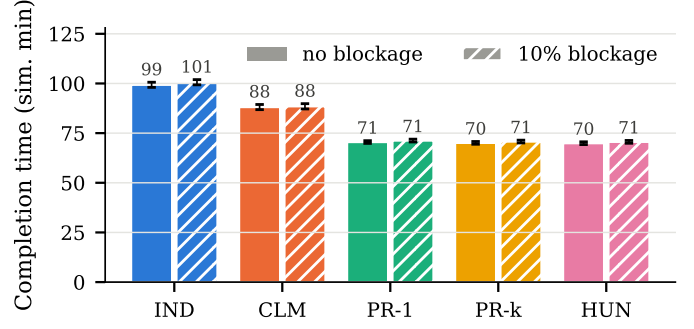


Fig. 2. E1 completion time per protocol with 95% confidence intervals (1,000 paired worlds). Solid bars: no blockage; hatched bars: 10% of road cells blocked mid-run. The ladder improves monotonically, with nearly all of the gain realized by PR-1.

TABLE III
E1 PAIRED COMPARISONS OF COMPLETION TIME ($b = 0$)

Step	ΔT [95% CI]	%	W/T/L	r_{rb}	p_{Holm}	% $\bar{w}_s$
IND→CLM	11.2 [10.3, 12.1]	11.3	581/385/34	0.96	$< 10^{-4}$	5.3
CLM→PR-1	17.6 [16.5, 18.7]	20.0	741/227/32	0.99	$< 10^{-4}$	18.7
IND→PR-1	28.8 [27.8, 29.9]	29.0	972/4/24	1.00	$< 10^{-4}$	23.0
PR-1→PR-k	0.4 [0.4, 0.5]	0.6	436/561/3	0.97	$< 10^{-4}$	0.8
PR-1→HUN	0.6 [0.4, 0.7]	0.8	452/518/30	0.79	$< 10^{-4}$	1.1
PR-k→HUN	0.2 [0.0, 0.3]	0.2	63/910/27	0.31	0.019	0.3

ΔT : mean paired reduction (min). W/T/L: worlds where the second protocol is faster / tied / slower. % $\bar{w}_s$: reduction in severity-weighted wait.

With negotiation, completion time tracks the ideal T_1/N curve closely: eight agents are 5.0–6.7× faster than one (63–84% parallel efficiency). Without coordination, eight agents are only 1.7–2.5× faster, and doubling the team from four to eight buys 3–12%. The mechanism is visible in Fig. 4(a): independent agents herd onto the same high-priority victims, and wasted effort grows from 241 agent-minutes at $N = 2$ to 1,662 at $N = 8$ ($M = 20$). Consequently the empirical price of anarchy rises almost linearly with team size, from 1.26–1.34 at $N = 2$ to 2.80–3.06 at $N = 8$, and PR-1 beats IND on 2,249 of the 2,250 paired worlds with $N \geq 2$, the remaining one being a tie (Holm-corrected $p < 10^{-20}$ in every cell). For a dispatcher, the practical reading is blunt: buying more vehicles without a coordination layer delivers a fraction of the capacity paid for.

C. E2: What Negotiation Adds, and When

The negotiation share η in Table IV is not a constant. It rises with the ratio of agents to victims: 4–11% at $N = 2$, 13–34% at $N = 4$, and 23–50% at $N = 8$, reaching 61% in the E1 setting ($M/N = 3$). When victims vastly outnumber agents, agents rarely become idle in the same tick with the same favourite, so public claims prevent almost all duplication and CLM is nearly as good as PR-1. As contention rises, simultaneous choices collide, claims arrive too late, and the proposal round becomes the decisive ingredient. The design rule is therefore conditional: claim broadcasting is sufficient for sparse, large-area operations, while dense incidents with many units need explicit negotiation.

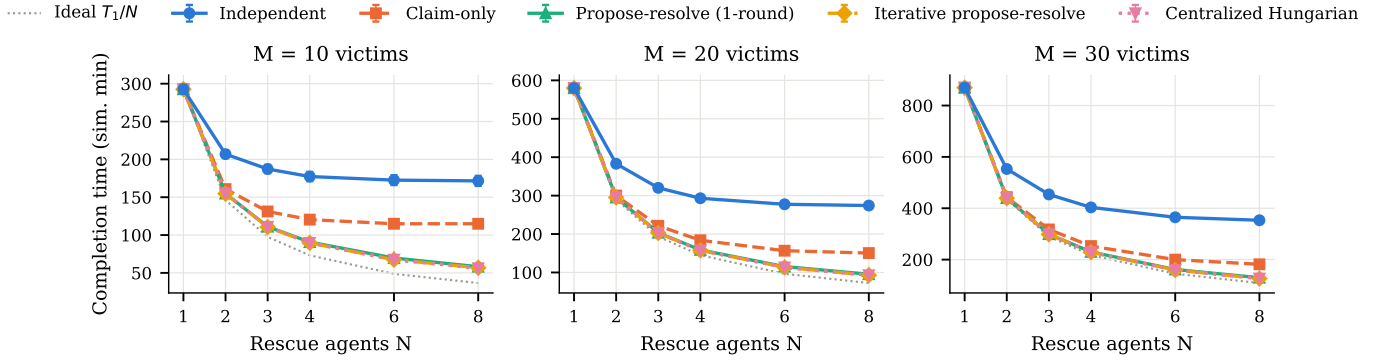


Fig. 3. E2: completion time versus team size N for three workloads (150 paired worlds per point, 95% CIs). The dotted grey curve is ideal linear speedup from the single-agent time. Independent agents (blue) flatten after $N \approx 4$; claim broadcasting (orange) recovers part of the gap; the three negotiating protocols overlap and stay close to the ideal curve.

TABLE IV
E2: SCALING, OPTIMALITY GAP, NEGOTIATION SHARE, AND COMMUNICATION VOLUME

M	N	Completion time T (min)				Gap to HUN (%)				Speedup $S(N)$			Payload
		IND	CLM	PR-1	HUN	PoA	PR-1	PR-K	η (%)	IND	PR-1	PR-1/HUN	
10	2	207	161	155	155	1.34	-0.0	-0.2	11	1.41	1.89	31/132	
10	4	177	120	91	90	1.98	+1.2*	-0.0	34	1.65	3.23	41/145	
10	8	172	115	58	56	3.06	+3.7*	+1.2	50	1.71	5.03	68/193	
20	2	383	300	295	296	1.30	-0.1	-0.1	6	1.51	1.96	61/462	
20	4	293	184	159	158	1.86	+0.7*	-0.0	19	1.98	3.65	72/475	
20	8	274	150	95	93	2.96	+2.9*	+0.1	31	2.11	6.07	118/532	
30	2	553	443	439	440	1.26	-0.1	-0.1	4	1.57	1.98	91/993	
30	4	403	253	230	229	1.76	+0.6*	+0.1	13	2.16	3.78	103/1006	
30	8	353	182	130	126	2.80	+2.5*	+0.0	23	2.46	6.71	152/1065	

PoA = $T_{\text{IND}}/T_{\text{HUN}}$. Gap: $(T_X - T_{\text{HUN}})/T_{\text{HUN}}$; * marks Holm-corrected Wilcoxon $p < 0.05$ (45 tests); negative values mean the protocol beat the per-epoch optimum. η : negotiation share, Eq. (3). Payload: scalars transmitted per run. 150 paired worlds per row.

D. E2: Decentralized Negotiation versus Centralized Optimization

PR-K is statistically indistinguishable from the centralized Hungarian matcher in all 15 (N, M) cells after correction, with mean gaps between -0.2% and $+1.2\%$. The simpler PR-1 is within 3.7% everywhere, but its gap grows with N and is significant in every cell with $N \geq 4$ (and in one of the three $N = 3$ cells): each conflict loser idles for one tick, and with more agents there are more losers per tick. The iterative retry of PR-K removes exactly this cost. The two negotiation protocols also move far less data than HUN, whose agents must report a utility for every candidate victim: at $M = 30$, PR-1 transmits 91–152 scalars per mission against roughly 1,000 for HUN. Because HUN optimizes only the current epoch, the decentralized protocols occasionally beat it (negative gaps at $N = 2$).

Fig. 4(b) shows a side effect worth noting. Coordinated teams accumulate *more* idle agent-minutes than independent ones (230 vs. 92 at $N = 8$, $M = 20$), because once every remaining victim is claimed, the surplus agents correctly wait instead of duplicating. That idle time is uncommitted capacity a dispatcher can redeploy, for example to reconnaissance or to a second incident, whereas the independent team's wasted

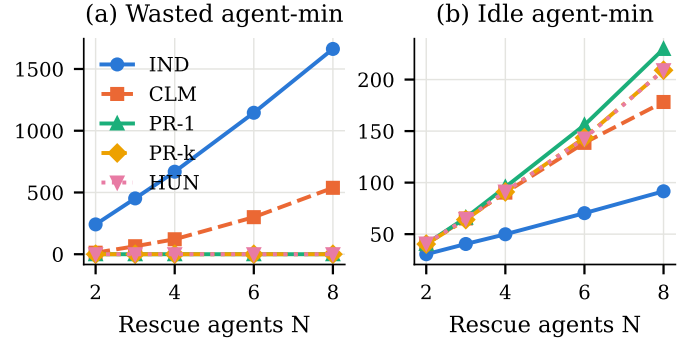


Fig. 4. E2 ($M = 20$): (a) agent-minutes spent on targets later abandoned; (b) agent-minutes idle while victims remained. Negotiation converts wasted travel into idle, redeployable capacity.

minutes cannot be recovered.

E. E3: Robustness to Message Loss

Fig. 5 shows that no protocol collapses under broadcast loss. Completion time rises smoothly from the loss-free value toward the IND level, which all protocols reach exactly at $p = 1$, as the model requires. Negotiation retains roughly a $(1 - p)$ fraction of its gain over IND: 92% at $p = 0.1$,

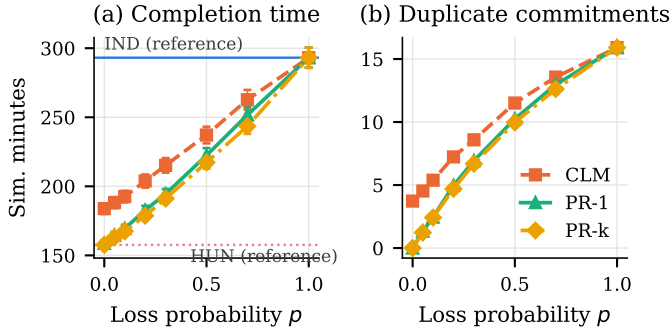


Fig. 5. E3 ($N = 4$, $M = 20$, 150 paired worlds per point): (a) completion time and (b) duplicate commitments versus broadcast-loss probability. Horizontal lines give the loss-free IND and HUN references. All protocols converge to IND at $p = 1$.

TABLE V
E4: COMPLETION TIME (MIN) UNDER ROAD BLOCKAGES ($N = 4$, $M = 20$)

b	IND	CLM	PR-1	PR-K	HUN	Gain (%)
0%	293.1	183.9	158.7	157.6	157.6	45.9
10%	294.9	184.7	160.1	158.9	158.9	45.7
20%	296.2	187.0	161.9	160.9	160.7	45.3
30%	299.5	187.5	163.8	162.6	162.5	45.3

Gain: IND $\rightarrow$ PR-1 completion-time reduction. 150 paired worlds per row.

73% at $p = 0.3$, and 53% at $p = 0.5$ for PR-1. With 10% of messages lost, PR-1 is still 42% faster than independent agents. The margin of negotiation over claim-only shrinks with loss (25.2 minutes at $p = 0$, 14.7 at $p = 0.5$), because a lost proposal becomes a unilateral commitment. PR-K gains relative to PR-1 as loss grows (8.2 minutes at $p = 0.7$), since its within-tick retries give a losing agent another chance to be heard. Duplicate commitments grow almost linearly in p (Fig. 5b), so the duplicate rate is a usable online indicator of link quality.

F. E4: Robustness to Road Blockages

Table V shows that dynamic blockages of up to 30% of road cells lengthen missions by at most 3.2% under any protocol and leave the IND $\rightarrow$ PR-1 gain essentially constant at 45–46%. Blockages affect routing, which every protocol handles with the same BFS replanning, but not the allocation question that separates them.

VI. DISCUSSION

A. Implications for Dispatch System Design

Three practical rules follow. *First, coordinate before you scale*: without a coordination layer the marginal value of each added unit falls quickly, so fleet expansion and coordination infrastructure should be planned together. *Second, match the protocol to contention*: when units are few relative to casualties, lightweight claim broadcasting (a status message per dispatch) captures most of the benefit; in dense incidents, add

a proposal round. *Third, centralization is not required for near-optimal allocation*: iterative propose–resolve matches a central optimizer with a fraction of the traffic and without a single point of failure, which matters when the operations center itself may be unreachable. These rules apply to ambulance and drone dispatch, warehouse and delivery robot fleets, and any setting where independent agents choose from a shared pool of tasks.

B. Comparison with Existing Work

Our near-equivalence between iterative negotiation and optimal assignment is consistent with the near-optimality of greedy auctions summarized by Quinton *et al.* [3], and our graceful-degradation result under message loss agrees qualitatively with Otte *et al.* [12] and with the 2026 benchmark of Lott and Honary [14]. Those studies compare allocators within the auction and consensus family. What they do not provide, and this one does, is a decomposition of the gain from no communication up to full centralization on a single codebase, which shows that the value of explicit negotiation depends on contention rather than being universal, and that without coordination, adding agents stops paying off.

C. Limitations

- The world is a synthetic grid with uniform travel cost, one hospital, one kit per victim, and all victims known at $t = 0$. Victim discovery, heterogeneous vehicles, and capacities above one are not modelled.
- The duplicated-effort penalty depends on simulator rules, in particular the 40-tick kit timeout and the “already allocated” kit denial. Different rules would change the absolute size of the IND penalty, although not the fact that duplicates waste effort.
- HUN is a per-epoch optimum, not a full-horizon optimum; the optimality gaps are relative to that reference.
- The loss model is independent broadcast erasure. Correlated outages, partitions, and delay are not modelled.
- Times are simulated minutes produced by the model, not field measurements, and priority weights (λ , β) were fixed rather than tuned.

D. Future Work

- Add staggered victim discovery by a sensing agent, which couples information latency with allocation.
- Add consensus-based bundle auctions with partial replanning [4] and learned bigraph matching [11] to the ladder, including multi-victim capacity.
- Replace independent loss with bursty (Gilbert–Elliott) loss, partitions, and delay [13], [14].
- Use the duplicate rate observed online to switch adaptively between claim-only and negotiation modes.

VII. CONCLUSION

We asked how much communication a rescue team needs, and answered it with a five-rung communication ladder evaluated on 28,300 paired simulations. Coordination is worth

29% of mission time for two agents and up to 66% for eight; without it, teams saturate at $1.7\text{--}2.5\times$ single-agent speed regardless of size. The value splits between broadcasting claims and negotiating simultaneous choices in a ratio set by contention, and a fully decentralized iterative negotiation matches a centralized optimal matcher while sending $3\text{--}11\times$ less data and degrading gracefully as messages are lost. For practitioners the message is simple: the first few bytes of coordination are worth more than the next few vehicles.

REPRODUCIBILITY

Code, fixed seeds, all raw per-run CSV files, and the scripts that generate every table and figure are in the RESQ-MAS repository (`experiments/study.py`, `experiments/paper_figures.py`). The 73-test suite covers every protocol, the loss model, and the optimal matcher.

REFERENCES

- [1] J. Peña Queraltá, J. Taipalmaa, B. Can Pullinen, V. K. Sarker, T. Nguyen Gia, H. Tenhunen, M. Gabbouj, J. Raitoharju, and T. Westerlund, “Collaborative multi-robot search and rescue: Planning, coordination, perception, and active vision,” *IEEE Access*, vol. 8, pp. 191617–191643, 2020.
- [2] D. S. Drew, “Multi-agent systems for search and rescue applications,” *Curr. Robot. Rep.*, vol. 2, pp. 189–200, 2021.
- [3] F. Quinton, C. Grand, and C. Lesire, “Market approaches to the multi-robot task allocation problem: A survey,” *J. Intell. Robot. Syst.*, vol. 107, no. 2, art. 29, 2023.
- [4] N. Buckman, H.-L. Choi, and J. P. How, “Partial replanning for decentralized dynamic task allocation,” in *Proc. AIAA SciTech Forum*, San Diego, CA, USA, 2019, paper AIAA 2019-0915.
- [5] S. Chopra, G. Notarstefano, M. Rice, and M. Egerstedt, “A distributed version of the Hungarian method for multirobot assignment,” *IEEE Trans. Robot.*, vol. 33, no. 4, pp. 932–947, 2017.
- [6] H. Chakraa, F. Guérin, E. Leclercq, and D. Lefebvre, “Optimization techniques for multi-robot task allocation problems: Review on the state-of-the-art,” *Robot. Auton. Syst.*, vol. 168, art. 104492, 2023.
- [7] H. Aziz, A. Pal, A. Pourmiri, F. Ramezani, and B. Sims, “Task allocation using a team of robots,” *Curr. Robot. Rep.*, vol. 3, pp. 227–238, 2022.
- [8] Athira K. A., Divya Udayan J., and U. Subramaniam, “A systematic literature review on multi-robot task allocation,” *ACM Comput. Surv.*, vol. 57, no. 3, art. 68, 2024.
- [9] E. Nunes, M. Manner, H. Mitiche, and M. Gini, “A taxonomy for task allocation problems with temporal and ordering constraints,” *Robot. Auton. Syst.*, vol. 90, pp. 55–70, 2017.
- [10] Y. Rizk, M. Awad, and E. W. Tunstel, “Cooperative heterogeneous multi-robot systems: A survey,” *ACM Comput. Surv.*, vol. 52, no. 2, art. 29, 2019.
- [11] P. Ghassemi and S. Chowdhury, “Multi-robot task allocation in disaster response: Addressing dynamic tasks with deadlines and robots with range and payload constraints,” *Robot. Auton. Syst.*, vol. 147, art. 103905, 2022.
- [12] M. Otte, M. J. Kuhlman, and D. Sofge, “Auctions for multi-robot task allocation in communication limited environments,” *Auton. Robots*, vol. 44, no. 3, pp. 547–584, 2020.
- [13] J. Gielis, A. Shankar, and A. Prorok, “A critical review of communications in multi-robot systems,” *Curr. Robot. Rep.*, vol. 3, pp. 213–225, 2022.
- [14] J. Lott and V. Honary, “Decentralized multi-robot task allocation under degraded communication: A benchmark of performance, reliability, and computation,” arXiv:2609.13711, 2026.
- [15] D. Paccagnan, R. Chandan, and J. R. Marden, “Utility design for distributed resource allocation—Part I: Characterizing and optimizing the exact price of anarchy,” *IEEE Trans. Autom. Control*, vol. 65, no. 11, pp. 4616–4631, 2020.
- [16] P. Virtanen *et al.*, “SciPy 1.0: Fundamental algorithms for scientific computing in Python,” *Nature Methods*, vol. 17, pp. 261–272, 2020.